

Detailed explanation of myblockly building blocks

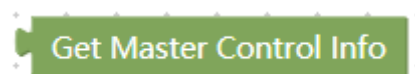
System Status

Get Basic firmware version



- **Prototype:** `get_basic_version()`
- **Interface Description:** Get the Basic firmware version.
- **Return:** Return to Basic firmware version.

Get Master Control Info



- **Prototype:** `get_system_version()`
- **Interface Description:** Get main control version information.
- **Return:** Return to Basic firmware version.

Get Robot ID



- **Prototype:** `get_robot_id()`
- **Interface Description:** Get the robot id.
- **Return:** Return the robot id.

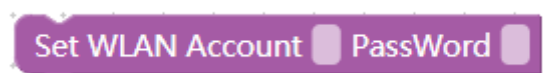
Set Robot ID



- **Prototype:** `set_robot_id(id)`
- **Interface Description:** Set the robot id.
- **Parameters:** -robot id

WLAN Setting

Set WLAN account password



- **Prototype:** `set_ssaid_pwd(account,password)`

- **Interface Description:** Set WLAN account password.
- Parameters
- :
 - wlan account
 - wlan password

Get WLAN Account and Password

Get WLAN Account and PassWord

- **Prototype:** `get_ssid_pwd()`
- **Interface Description:** Get the WLAN account password.
- Return
- :
 - wlan account
 - wlan password

Set Server Port

Set Server Port

- **Prototype:** `set_server_port(port)`
- **Interface Description:** Set the port for WLAN connection.
- Parameters
- :
 - The port number

Basic

Set basic pin output

Set basic pin 1 output 0

- **Prototype:** `set_basic_output(id,state)`
- **Interface Description:** Set base pin output
- Parameters
- :
 - Pin number
 - Select output status

Get basic pin input



- **Prototype:** `set_basic_output(id)`
- **Interface Description:** Get base pin input
- Return
 - Base pin input status

ATOM Io

Set Color



- **Prototype:** `set_color(r=0,g=0,b=0)`
- **Interface Description:** Set the color of the end LED light.
- **Parameters**
 - red
 - green
 - blue

Set PWM output



- **Prototype:** `set_pwm_output(v1=0,v2=0,v3=0)`
- **Interface Description:** Set PWM output.
- **Parameters**
 - **v1** (*int*)
 - **v2** (*int*)
 - **v3** (*int*)

Set end pin mode



- **Prototype:** `set_pin_mode(id,state)`
- **Interface Description:** Set the end pin mode.
- **Parameters**
 - Pin number
 - Select mode

Set IO value

A green block with a notch on the left side. It contains the text "setting IO value" followed by a dropdown menu showing "1" and "value" followed by a dropdown menu showing "0".

- **Prototype:** `set_digital_output(id, state)`
- **Interface Description:** Set IO value
- **Parameters**
 - io serial number
 - Select status 0 or 1

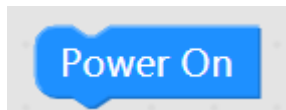
Read IO value

A green block with a pin on the left side. It contains the text "Get IO value" followed by a dropdown menu showing "1".

- **Prototype:** `get_digital_input(id)`
- **Interface Description:** Read IO value
- **Parameters**
 - io serial number

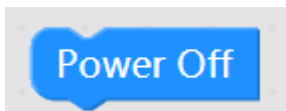
Status

Power On

A blue block with a notch on the left side. It contains the text "Power On".

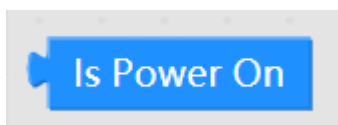
- **Prototype:** `power_on()`
- **Interface Description:** Atom opens communication (open by default).

Power Off

A blue block with a pin on the left side. It contains the text "Power Off".

- **Prototype:** `power_off()`
- **Interface Description:** Atom closes communication.

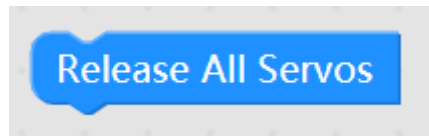
Check whether the robot arm is powered on

A blue block with a pin on the left side. It contains the text "Is Power On".

- **Prototype:** `is_power_on()`
- **Interface Description:** Check whether the robot arm is powered on.
- **Return:**
 - `1`: power on

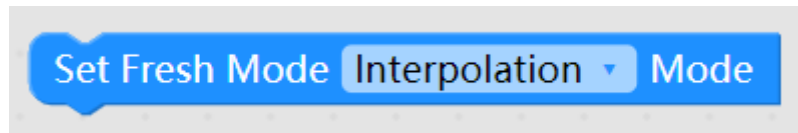
- 0: power off
- -1: error

release all Servos



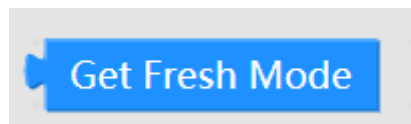
- **Prototype:** `release_all_servos()`
- **Interface Description:** Set the robot arm to free movement mode

Set command refresh mode



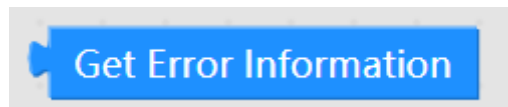
- **Prototype:** `set_fresh_mode(mode)`
- **Interface Description:** Set command refresh mode
- **Parameters**
 - **mode:** Refresh Sport - always execute the latest command first. Interpolation - Execute instructions sequentially in a queue.

Get command refresh mode



- **Prototype:** `get_fresh_mode()`
- **Interface Description:** Get command refresh mode
- **Return**
 - 1 - Always execute the latest command first.
 - 0 - Execute instructions sequentially in a queue.

Get Error Infomation



- **Prototype:** `get_error_information()`
- **Interface Description:** Get robot error information.
- **Return:**
 - 0: No error.
 - 1 ~ 6: The corresponding joint exceeds the limit position.
 - 32: The inverse kinematics solution has no solution.
 - 33 ~ 34: There is no adjacent solution for linear motion.

Clear Error Information

Clear Error Information

- **Prototype:** `clear_error_information()`
- **Interface Description:** Clear robot error messages.

Check if Atom is connected

Is Controller Connected

- **Prototype:** `is_controller_connected()`
- **Interface Description:** Check whether Atom is connected
- Return
 - :
 - 0: Not connected
 - 1: Connected

Set free mode

Set Free Mode Status Open ▾

- **Prototype:** `set_free_mode(state)`
- **Interface Description:** Whether to set to free mode
- Parameters
 - :
 - **state**(int): 0 | 1

Is free mode

Is Free Mode

- **Prototype:** `is_free_mode()`
- **Interface description:** Check whether it is free mode
- return value
 - :
 - **state**(int): 0 | 1

MDI Control

Get Angles

Get Angles

- **Prototype:** `get_angles()`

- **Interface Description:** Get the degrees of all joints.
- Return
 - joint angle value

Get Coords



- **Prototype:** `get_coords()`
- **Interface Description:** Get the Cartesian coordinates of the machine.
- Return
 - Cartesian coordinates

Set single joint angle



- **Prototype:** `send_angle(id, degree, speed)`
- **Interface Description:** Robot single joint angle control.
- **Parameters**
 - Joint id: 1-6
 - Angle value
 - speed

Set single coordinates



- **Prototype:** `send_coord(id, value, speed)`
- **Interface Description:** Robot single coordinate control.
- **Parameters**
 - Coordinates: 1-6 correspond to x, y, z, rx, ry, rz
 - coordinate value
 - speed

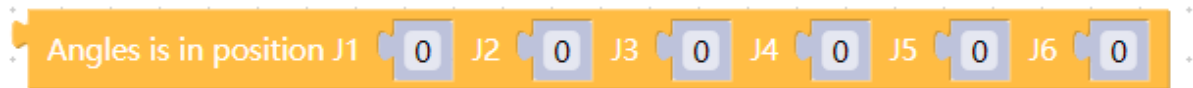
Check if you are in motion



- **Prototype:** `is_moving(id, value, speed)`
- **Interface Description:** Detect whether the machine is moving.

- Return
 - :
 - 0: The machine is not in motion
 - 1: The machine is in motion.

Angles is in position



- **Prototype:** `is_in_position([j1,j2,j3,j4,j5,j6],0)`
- **Interface description:** Check whether the machine angle reaches the specified position
- **Parameters:**
 - j1 angle value
 - j2 angle value
 - j3 angle value
 - j4 angle value
 - j5 angle value
 - j6 angle value
- **Return:**
 - 0: Not reached the specified location
 - 1: Reach the specified location

Coords is in position



- **Prototype:** `is_in_position([x,y,z,rx,ry,rz],1)`
- **Interface description:** Check whether the machine angle reaches the specified position
- **Parameters:**
 - x coordinate value
 - y coordinate value
 - z coordinate value
 - rx coordinate value
 - ry coordinate value
 - rz coordinate value
- **Return:**
 - 0: Not reached the specified location
 - 1: Reach the specified location

Set Angles



- **Prototype:** `send_angles(angles, speed)`
- **Interface Description:** Send all angles to the robot arm.
- **Parameters**
 - `angles`: list of coordinate values (`List[float]`).
 - `speed`: (`int`) 0 ~ 100

Set Coords



- **Prototype:** `send_coords(coords, speed, mode)`
- **Interface Description:** Send all coordinates to the robot arm.
- **Parameters**
 - `coords`: list of coordinate values (`List[float]`).
 - `speed`: (`int`) 0 ~ 100
 - `mode`: (`int`): 0 - angular (default), 1 - linear

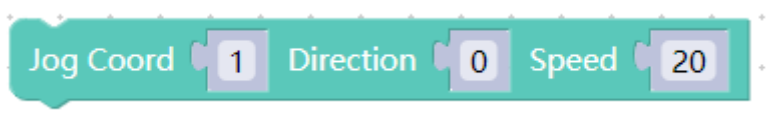
JOG Control

JOG Angle



- **Prototype:** `jog_angle(joint_id, direction, speed)`
- **Interface description:** Jog control angle
- **Parameters**
 - `joint_id`: (`int`) 1 ~ 6
 - `direction`: 0 - decrease, 1 - increase
 - `speed`: 0 ~ 100

JOG Coord



- **Prototype:** `jog_coord(coord_id, direction, speed)`
- **Interface description:** Jog control coord.

- **Parameters**

- `coord_id:(int)` 1 ~ 6
- `direction:` 0 - decrease, 1 - increase
- `speed:` 0 ~ 100

JOG Absolute



- **Prototype:** `jog_absolute(coord_id, direction, speed)`

- **Interface description:** Step mode.

- **Parameters**

- `coord_id:(int)` 1 ~ 6
- `direction:`
- `speed:` 0 ~ 100

JOG Increment



- **Prototype:** `jog_increment(coord_id, direction, speed)`

- **Interface description:** Step mode.

- **Parameters**

- `coord_id:(int)` 1 ~ 6
- `direction:`
- `speed:` 0 ~ 100

Jog Stop



- **Prototype:** `jog_stop()`

- **Interface Description:** Stop jog moving.

#

Pause



- **Prototype:** `pause()`

- **Interface Description:** Pause movement.

Resume

A teal rectangular button with rounded corners and a slight shadow, containing the word "Resume" in white text.

- **Prototype:** `resume()`
- **Interface description:** Recovery movement.

Stop

A teal rectangular button with rounded corners and a slight shadow, containing the word "Stop" in white text.

- **Prototype:** `stop()`
- **Interface Description:** Stop moving.

is_paused

A teal rectangular button with rounded corners and a slight shadow, containing the text "Is Paused" in white text.

- **Prototype:** `is_paused()`
- **Interface Description:** Judge whether the manipulator pauses or not.
- **Return:** :
 - `1` - paused
 - `0` - not paused
 - `-1` - error

Set Servo encoder value

A teal rectangular button with rounded corners and a slight shadow. It contains the text "Set servo" followed by a small purple square with the number "1" inside, then the text "encoder value" followed by another small purple square with the number "0" inside.

- **Prototype:** `set_encoder(joint_id, encoder)`
- **Interface Description:** Set a single joint rotation to the specified potential value.
- **Parameters**
 - `joint_id`: (`int`) 1 ~ 6
 - `encoder`: 0 ~ 4096

Get Servo encoder value

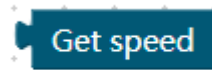
A teal rectangular button with rounded corners and a slight shadow. It contains the text "Get servo" followed by a small purple square with the number "1" inside, then the text "encoder value".

- **Prototype:** `get_encoder(joint_id)`
- **Interface description:** Obtain the specified joint potential value.
- **Parameters:** `joint_id`: (`int`) 1 ~ 6

- **Return:** `encoder`: 0 ~ 4096

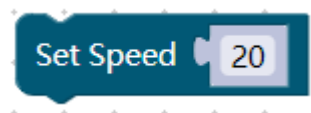
Setting

get_speed



- **Prototype:** `get_speed()`
- **Interface description:** Get speed.
- **Return:** speed: (`int`)

set_speed



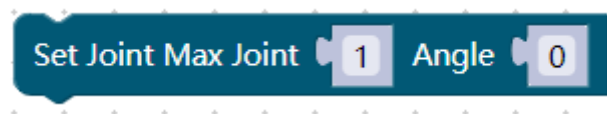
- **Prototype:** `set_speed(speed)`
- **Interface description:** Set speed.
- **Parameters:** speed: (`int`) 0 ~ 100

set_joint_min



- **Prototype:** `set_joint_min(id, angle)`
- **Interface description:** Sets the minimum angle for the specified joint.
- **Parameters:**
 - `id`: (`int`) joint id 1-6.
 - `angle`: 0 - 180.

set_joint_max



- **Prototype:** `set_joint_max(id, angle)`
- **Interface description:** Sets the maximum angle of the specified joint.
- **Parameters:**
 - `id`: (`int`) joint id 1-6.
 - `angle`: 0 - 180.

get_joint_min_angle



- **Prototype:** `get_joint_min_angle()`
- **Interface description:** Gets the minimum movement angle of the specified joint
- **Parameters:** `joint_id: (int)`
- **Return:** angle value (`float`)

get_joint_max_angle



- **Prototype:** `get_joint_max_angle()`
- **Interface description:** Gets the maximum movement angle of the specified joint
- **Parameters:** `joint_id: (int)`
- **Return:** angle value (`float`)

Servo control

is_servo_enable



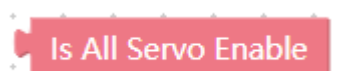
- **Prototype:** `is_servo_enable(servo_id)`
- **Interface description:** Determine whether all steering gears are connected
- **Parameters:** `servo_id (int) 1 ~ 6`
- **Return:**

- 0

: disable

- 1: enable
- -1: error

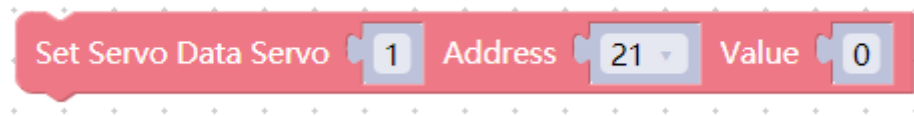
is_all_servo_enable



- **Prototype:** `is_all_servo_enable()`
- **Interface description:** Determine whether the specified steering gear is connected
- **Return:**

- 0: disable
- 1: enable
- -1: error

set_servo_data



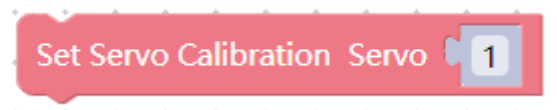
- **Prototype:** `set_servo_data(servo_no, data_id, value)`
- **Interface description:** Set the data parameters of the specified address of the steering gear.
- **Parameters:**
 - `servo_no`: Serial number of articulated steering gear, 1 - 6.
 - `data_id`: Data address.
 - `value`: 0 - 4096

get_servo_data



- **Prototype:** `get_servo_data(servo_no, data_id)`
- **Interface description:** Read the data parameter of the specified address of the steering gear.
- **Parameters:**
 - `servo_no`: Serial number of articulated steering gear, 1 - 6.
 - `data_id`: Data address.
- **Return:** `value`: 0 - 4096
 - 0: disable
 - 1: enable
 - -1: error

set_servo_calibration



- **Prototype:** `set_servo_calibration(servo_no)`
- **Interface description:** The current position of the calibration joint actuator is the angle zero point, and the corresponding potential value is 2048.
- **Parameters:**
 - `servo_no`: Serial number of articulated steering gear, 1 - 6.

release_servo



- **Prototype:** `release_servo(servo_id)`
- **Interface description:** Power off designated servo
- **Parameters:** `servo_id`: 1 ~ 6

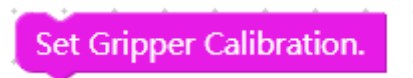
focus_servo



- **Prototype:** `focus_servo(servo_id)`
- **Interface description:** Power on designated servo
- **Parameters:** `servo_id`: 1 ~ 6

Gripper

set_gripper_calibration



- **Prototype:** `set_gripper_calibration()`
- **Interface description:** Set the current position to zero, set current position value is `2048`.

set_gripper_state



- **Prototype:** `set_gripper_state(flag, speed, mode)`
- **Interface description:** Set gripper switch state
- **Parameters**
 - `flag` (`int`): 0 - open, 1 - close
 - `speed` (`int`): 0 ~ 100
 - `mode`: gripper

set_gripper_value

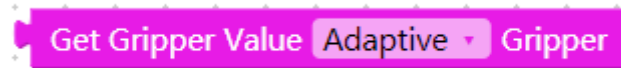


- **Prototype:** `set_gripper_value(value, speed, mode)`
- **Interface description:** Set gripper value

- **Parameters**

- `value` (int): 0 ~ 100
- `speed` (int): 0 ~ 100
- `mode`: gripper

get_gripper_value



- **Prototype:** `get_gripper_value(mode)`
- **Interface description:** Get gripper value
- **Return:** gripper value (int)

is_gripper_moving



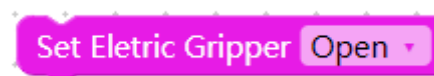
- **Prototype:** `is_gripper_moving()`
- **Interface description:** Judge whether the gripper is moving or not
- **Return:**
 - `0` : not moving
 - `1` : is moving
 - `-1` : error data

Init Eletric Gripper



- **Prototype:** `init_eletric_gripper()`
- **Interface description:** Init Eletric Gripper

Set Eletric Gripper



- **Prototype:** `set_eletric_gripper()`
- **Interface description:** Init Eletric Gripper
- **Parameters**
 - 选择打开或关闭

set_gripper_mode

Set Gripper Mode Transparent Transmission ▾

- **Prototype:** `set_gripper_mode(status)`
- **Interface description:** Set gripper mode.
- **Parameters**
- `status` (`int`): 0 - transparent transmission. 1 - Port Mode.

get_gripper_mode

Get Gripper Mode

- **Prototype:** `get_gripper_mode()`
- **Interface description:** Get gripper mode.
- **Return**
 - `status` (`int`): 0 - transparent transmission. 1 - Port Mode.

Coord Control

get_tool_reference

Get Tool coordinate system

- **Prototype:** `get_tool_reference()`
- **Interface description:** Get tool coordinate system.
- **Return:** `list` `[x, y, z, rx, ry, rz]`.

set_tool_reference

Set Tool Coord X 0 Y 0 Z 0 Rx 0 Ry 0 Rz 0

- **Prototype:** `set_tool_reference(coords)`
- **Interface description:** Set tool coordinate system.
- **Parameters:**
 - `coords`: (`list`) `[x, y, z, rx, ry, rz]`.

get_world_reference

Get World coordinate system

- **Prototype:** `get_world_reference()`
- **Interface description:** Get world coordinate system.

- **Return:** `list` `[x, y, z, rx, ry, rz]`.

set_world_reference

Set World Coord X `0` Y `0` Z `0` Rx `0` Ry `0` Rz `0`

- **Prototype:** `set_world_reference(coords)`
- **Interface description:** Set world coordinate system.
- **Parameters:**
 - `coords: (list)` `[x, y, z, rx, ry, rz]`.

get_reference_frame

Get Base Coord

- **Prototype:** `get_reference_frame()`
- **Interface description:** Get base coordinate system.
- **Return:** 0 - base 1 - tool.

set_reference_frame

Set Base Coord Base ▾

- **Prototype:** `set_reference_frame(rftype)`
- **Interface description:** Set base coordinate system.
- **Parameters:**
 - `rftype`: 0 - base 1 - tool.

get_movement_type

Get Movement Type

- **Prototype:** `get_movement_type()`
- **Interface description:** Get movement type.
- **Return:** 1 - moveI, 0 - moveJ.

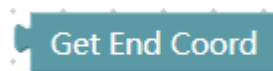
set_movement_type

Set Movement Type MoveI ▾

- **Prototype:** `set_movement_type(move_type)`
- **Interface description:** Set movement type.
- **Parameters:**

- `move_type`: 1 - moveI, 0 - moveJ.

get_end_type



- **Prototype:** `get_end_type()`
- **Interface description:** Get end coordinate system.
- **Return:** 0 - flange, 1 - tool.

set_end_type



- **Prototype:** `set_end_type(end)`
- **Interface description:** Set end coordinate system.
- **Parameters:**
 - `end`: 0 - flange, 1 - tool.

MyCobot320

get_servo_speeds



- **Prototype:** `get_servo_speeds()`
- **Interface description:** Get joint velocity.
- **Return:** `list` Speed of each joint.

get_servo_currents



- **Prototype:** `get_servo_currents()`
- **Interface description:** Get joint current.
- **Return:** `list` Current of each joint.

get_servo_voltages



- **Prototype:** `get_servo_voltages()`
- **Interface description:** Get joint voltage.

- **Return:** `list` Voltage of each joint.

get_servo_status



- **Prototype:** `get_servo_status()`
- **Interface description:** Get the state of each joint.
- **Return:** `list` the state of each joint.

get_servo_temps



- **Prototype:** `get_servo_temps()`
- **Interface description:** Get the temperature of each joint.
- **Return:** `list` temperature of each joint.